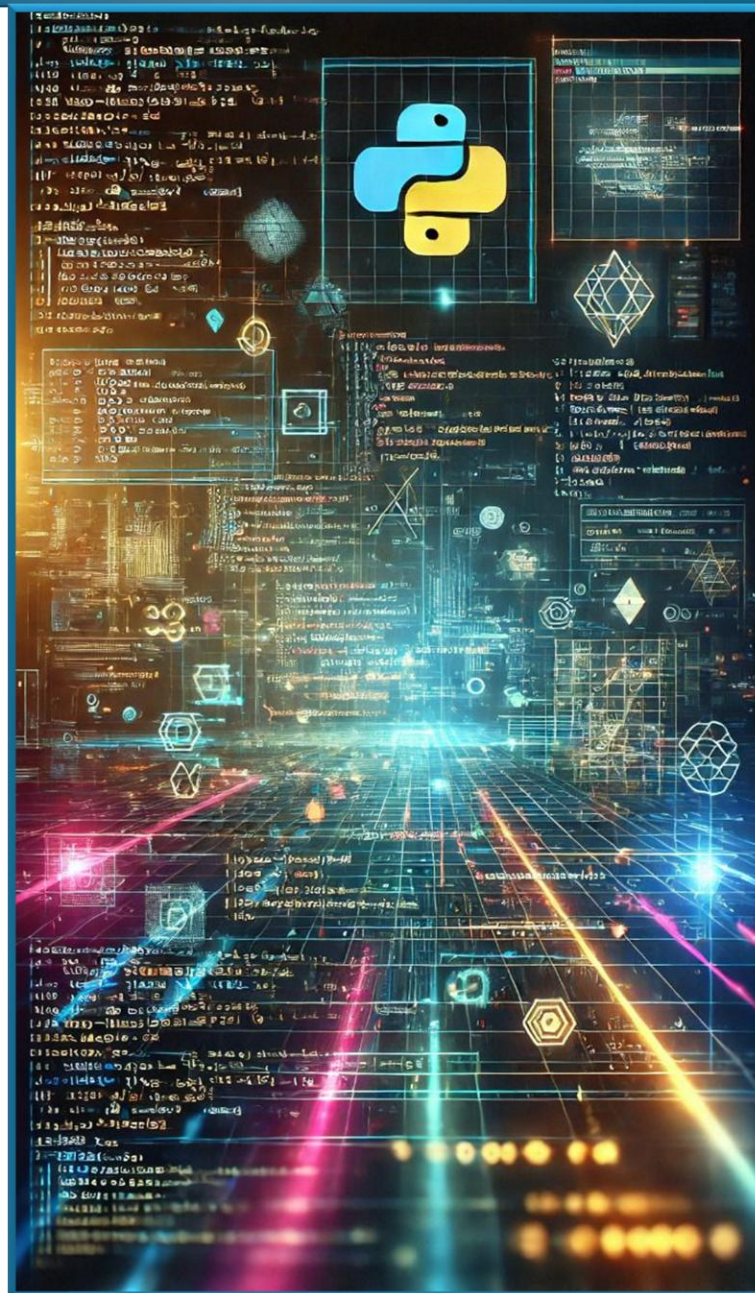


# 2026

## Buenas prácticas en el proceso de implementación de software



Ing. Carolina Vargas

Unidad 3 - Metodología de Sistemas II

---

## Contenido

|                                                                |    |
|----------------------------------------------------------------|----|
| Introducción .....                                             | 3  |
| Buenas prácticas de programación-Conceptos Fundamentales ..... | 3  |
| Código limpio (Clean Code) .....                               | 4  |
| Beneficios y consecuencias de su ausencia.....                 | 5  |
| Convenciones y estándares de codificación .....                | 5  |
| Estándares reconocidos por lenguaje .....                      | 6  |
| Automatización mediante linters y formateadores.....           | 6  |
| Documentación del código .....                                 | 7  |
| Tipos y niveles de documentación .....                         | 7  |
| Principios para una documentación efectiva .....               | 8  |
| Modularidad, reutilización y mantenibilidad .....              | 8  |
| Principios SOLID .....                                         | 9  |
| Reutilización .....                                            | 10 |
| Mantenibilidad.....                                            | 10 |
| Gestión de dependencias y paquetes.....                        | 10 |
| Tipos de Gestión de Dependencias .....                         | 13 |
| Buenas Prácticas de la Gestión de Dependencias.....            | 14 |
| Como gestionar dependencias .....                              | 14 |
| ¿Cómo gestionar las dependencias? .....                        | 15 |
| Gestores de Paquetes.....                                      | 15 |
| Gestores de Paquete .....                                      | 16 |
| Gestores de paquetes por ecosistema.....                       | 18 |
| Control de versiones.....                                      | 18 |
| Buenas prácticas de commits .....                              | 19 |
| Versionado semántico y reproducibilidad .....                  | 20 |
| Riesgos de seguridad y buenas prácticas.....                   | 20 |
| Estrategias de ramificación y flujos de trabajo .....          | 20 |
| Git Flow.....                                                  | 21 |
| Trunk-Based Development.....                                   | 21 |
| Modelos intermedios y criterios de selección .....             | 22 |
| Pull Requests y Merge Requests.....                            | 22 |
| Buenas prácticas en su elaboración.....                        | 23 |

|                                                                                                 |           |
|-------------------------------------------------------------------------------------------------|-----------|
| <b>Integración con la automatización de calidad .....</b>                                       | <b>23</b> |
| <b>Trabajo colaborativo y resolución de conflictos .....</b>                                    | <b>23</b> |
| <b>Coordinación en equipos de desarrollo.....</b>                                               | <b>24</b> |
| <b>Conflictos de fusión: causas y resolución .....</b>                                          | <b>24</b> |
| <b>Prácticas colaborativas complementarias .....</b>                                            | <b>24</b> |
| <b>Refactorización continua .....</b>                                                           | <b>25</b> |
| <b>Señales que motivan la refactorización.....</b>                                              | <b>25</b> |
| <b>La necesidad de una red de seguridad de pruebas .....</b>                                    | <b>26</b> |
| <b>Revisión de código (Code Review) .....</b>                                                   | <b>26</b> |
| <b>Buenas prácticas para quienes revisan .....</b>                                              | <b>27</b> |
| <b>Buenas prácticas para quienes reciben la revisión.....</b>                                   | <b>27</b> |
| <b>Introducción al aseguramiento de la calidad del software durante la implementación .....</b> | <b>27</b> |
| <b>La calidad como proceso continuo .....</b>                                                   | <b>27</b> |
| <b>12.3 Integración y despliegue continuos .....</b>                                            | <b>28</b> |
| <b>Métricas y análisis estático .....</b>                                                       | <b>29</b> |
| <b>Conclusión .....</b>                                                                         | <b>29</b> |
| <b>Estándares de Codificación .....</b>                                                         | <b>30</b> |
| <b>Beneficios de aplicar estándares de Codificación .....</b>                                   | <b>30</b> |
| <b>Estándar de codificación abierto .....</b>                                                   | <b>30</b> |
| <b>Estándares de Codificación Cerrados .....</b>                                                | <b>31</b> |
| <b>Buenas Prácticas para implementar estándares de codificación .....</b>                       | <b>31</b> |
| <b>Convenciones de Codificación.....</b>                                                        | <b>33</b> |
| <b>Bibliografía .....</b>                                                                       | <b>35</b> |

---

## Buenas prácticas de programación y aseguramiento de la calidad del software durante la implementación

### Introducción

El desarrollo de software constituye una actividad compleja que requiere mucho más que la capacidad de escribir instrucciones en un lenguaje de programación. La calidad de un producto informático depende, en gran medida, de la forma en que se construye y mantiene su código fuente. Un sistema puede cumplir inicialmente con sus funcionalidades, pero si fue desarrollado sin criterios de organización, documentación y control, su evolución puede convertirse rápidamente en una tarea costosa y difícil.

En este contexto, las **buenas prácticas de programación** representan un conjunto de principios, técnicas y procedimientos orientados a mejorar la calidad del software.

Estas prácticas favorecen la legibilidad, la mantenibilidad, la reutilización y la confiabilidad del código, al mismo tiempo que facilitan el trabajo colaborativo entre los integrantes de un equipo de desarrollo.

Actualmente, el desarrollo profesional de software se apoya en metodologías y herramientas que permiten organizar el trabajo de manera eficiente. El uso de estándares de codificación, sistemas de control de versiones, estrategias de ramificación, revisiones de código y mecanismos de gestión de dependencias constituye una parte fundamental del proceso de implementación. Asimismo, actividades como la refactorización continua y el aseguramiento de la calidad permiten detectar y prevenir problemas antes de que lleguen a las etapas finales del desarrollo.

### Buenas prácticas de programación-Conceptos Fundamentales

Las buenas prácticas de programación pueden definirse como un:

**conjunto de recomendaciones y principios que orientan la producción de código fuente de manera ordenada, comprensible y sostenible.**



Su objetivo principal es facilitar tanto el desarrollo inicial como la posterior evolución del software.

Un programa no debe ser evaluado únicamente por su capacidad para ejecutarse correctamente. También resulta importante considerar si otras personas pueden comprenderlo, modificarlo, probarlo y ampliarlo sin introducir nuevos errores. Por esta razón, la calidad del código constituye un factor esencial en el desarrollo de sistemas.

Entre las principales buenas prácticas se pueden mencionar:

- Utilizar nombres claros y significativos.
- Mantener las funciones y métodos simples y con responsabilidades bien definidas.
- Evitar la duplicación innecesaria de código.
- Aplicar estándares y convenciones de codificación.
- Documentar adecuadamente los componentes relevantes.
- Realizar pruebas y revisiones de forma sistemática.
- Utilizar herramientas de control de versiones.
- Refactorizar el código para mejorar su estructura.

- 
- Favorecer el modularidad y la reutilización.

Estas prácticas no deben considerarse reglas aisladas, sino elementos complementarios de una cultura orientada a la calidad. Su aplicación permite reducir la complejidad del software y minimizar los riesgos asociados a su mantenimiento.

### Código limpio (Clean Code)

El término «código limpio» fue popularizado por Robert C. Martin en su obra de referencia publicada en 2008, donde reunió y sistematizó un conjunto de principios y heurísticas orientados a producir código legible, simple y fácil de mantener. Según esta perspectiva, un código limpio es aquel que puede ser leído y comprendido por otro programador con un esfuerzo mínimo, y que revela claramente la intención de quien lo escribió, sin necesidad de comentarios adicionales que expliquen «qué» hace cada línea.

Esta noción parte de una premisa central: el código se lee muchas más veces de las que se escribe. En consecuencia, optimizar exclusivamente por la velocidad de escritura inicial, en detrimento de la legibilidad futura, constituye una decisión que a largo plazo incrementa los costos de mantenimiento y la probabilidad de introducir errores.

El concepto de **Código Limpio o Clean Code** se refiere a la **escritura de programas que sean fáciles de leer, comprender y modificar.**

Un código limpio no solamente funciona correctamente, sino que expresa de manera clara la intención de quien lo desarrolló.

Uno de los principales referentes de este enfoque es Robert C. Martin, quien sostiene que el código debe ser entendido como una forma de comunicación entre desarrolladores. Los programas suelen permanecer activos durante largos períodos y pueden ser modificados por personas diferentes de quienes los escribieron originalmente. Por ello, la claridad resulta tan importante como la funcionalidad.

Algunos principios del código limpio son las siguientes:

- **Nombres significativos:** Las variables, funciones, clases y demás elementos deben tener nombres que expresen claramente su propósito. Por ejemplo, un nombre como `totalVentasMensuales` resulta más comprensible que una denominación genérica como `x` o `dato`.
- **Funciones pequeñas y con una responsabilidad definida:** Una función debe realizar una tarea concreta. Cuando un método acumula múltiples responsabilidades, aumenta su complejidad y se dificulta su comprensión y prueba.
- **Evitar la duplicación:** La repetición de código puede generar problemas de mantenimiento. Si una misma lógica se encuentra en diferentes lugares, cualquier modificación deberá realizarse en todos ellos. El principio conocido como **DRY (Don't Repeat Yourself)** propone evitar estas duplicaciones mediante una adecuada abstracción.

- 
- **Código legible:** La estructura del programa debe facilitar su lectura. Una correcta organización, una adecuada separación de responsabilidades y la utilización moderada de comentarios contribuyen a mejorar la comprensión.

A estos principios se suman el manejo explícito y consistente de errores mediante excepciones o estructuras de control claras, la reducción de los efectos secundarios ocultos en las funciones, y la preferencia por estructuras de control simples, evitando anidamientos profundos de condicionales y bucles que dificultan el seguimiento del flujo lógico del programa. La aplicación conjunta de estos principios contribuye a que el código funcione, en cierto sentido, como una forma de documentación viva del sistema.

### Beneficios y consecuencias de su ausencia

Los beneficios de escribir código limpio se manifiestan principalmente en el mediano y largo plazo: menor tiempo de incorporación de nuevos integrantes al equipo, reducción de la cantidad de defectos introducidos durante modificaciones, y mayor facilidad para realizar revisiones de código efectivas.

Por el contrario, la acumulación de código difícil de entender —frecuentemente denominada **deuda técnica**— genera un efecto de retroalimentación negativa: cada nueva funcionalidad se vuelve más costosa de implementar, lo que a su vez incentiva atajos que empeoran aún más la calidad del sistema.

Un exceso de abstracciones, capas o patrones de diseño aplicados sin necesidad real puede resultar tan perjudicial para la mantenibilidad como el código desordenado, dado que incrementa la carga cognitiva sin aportar un beneficio proporcional. La búsqueda del equilibrio entre simplicidad y flexibilidad constituye, por lo tanto, una habilidad central del desarrollador experimentado.

**El objetivo del código limpio no es alcanzar una perfección absoluta, sino construir software que pueda evolucionar de manera segura y eficiente.**

Los principios de código limpio descritos en este ítem constituyen, en gran medida, el fundamento sobre el cual se apoyan varias de las prácticas analizadas en los capítulos siguientes: un código legible y bien estructurado resulta más sencillo de documentar, de revisar durante el proceso de pull request y de refactorizar de manera incremental sin introducir errores.

### Convenciones y estándares de codificación

**Las convenciones de codificación son acuerdos o un conjunto de reglas acordadas por un equipo, una organización o una comunidad de lenguaje establecidos para mantener una forma uniforme de escribir código dentro de un proyecto o una organización.**



---

A diferencia del código limpio, que se ocupa principalmente de la calidad conceptual del diseño, las convenciones se centran en la uniformidad superficial del código.

La importancia de estas convenciones no radica tanto en cuál estilo particular se adopte, sino en que todo el equipo lo aplique de manera consistente. Cuando distintos desarrolladores utilizan estilos heterogéneos dentro de un mismo proyecto, se incrementa el esfuerzo cognitivo requerido para leer el código y aumenta la probabilidad de que las revisiones de código se centren en discusiones estilísticas en lugar de aspectos funcionales relevantes.

Incluyen aspectos relacionados con el formato, los nombres, la organización de archivos y otras decisiones de implementación.

Por ejemplo, un equipo puede definir convenciones para:

- Nombrar variables, funciones y clases.
- Organizar la estructura de los proyectos.
- Establecer el uso de sangrías y espacios.
- Determinar la longitud máxima de las líneas.
- Definir la ubicación de llaves y bloques de código.
- Establecer reglas para el manejo de errores.
- Longitud máxima de línea y la organización general de los archivos fuente.

La importancia de los estándares radica en que permiten que el código tenga una apariencia y estructura coherentes, independientemente de quién lo haya desarrollado. Esto reduce el esfuerzo necesario para comprender partes nuevas del sistema.

### **Estándares reconocidos por lenguaje**

Cada ecosistema de lenguaje de programación ha desarrollado sus propias guías de estilo ampliamente adoptadas. En Python, la guía PEP 8 establece convenciones oficiales sobre nombrado, espaciado e importaciones. En el ámbito de Java, las guías de estilo publicadas por Google constituyen una referencia habitual en entornos corporativos. En JavaScript y TypeScript, guías como la de Airbnb han influido significativamente en la configuración de herramientas de análisis estático utilizadas en la industria.

Estas guías no solo abordan cuestiones de formato, sino también recomendaciones sobre buenas prácticas específicas del lenguaje, como el uso adecuado de estructuras de datos, patrones de manejo de errores idiomáticos y convenciones de organización de módulos.

### **Automatización mediante linters y formateadores**

La adopción efectiva de convenciones de codificación en equipos de trabajo depende, en la práctica, de su automatización. Herramientas como ESLint y Prettier en el ecosistema JavaScript, Black y Pylint en Python, o Checkstyle en Java, permiten verificar y corregir automáticamente el cumplimiento de las reglas de estilo definidas, integrándose habitualmente en el editor de código y en los flujos de integración continua.

---

Esta automatización presenta una doble ventaja: por un lado, libera a los revisores de código de la tarea repetitiva de señalar cuestiones de formato, permitiéndoles concentrar su atención en aspectos de diseño y lógica; por otro, elimina buena parte de la subjetividad y las discusiones improductivas que suelen surgir cuando las normas de estilo dependen exclusivamente del criterio individual de cada desarrollador.

**En definitiva, las convenciones de codificación complementan al código limpio: mientras este último se ocupa de la calidad conceptual del diseño, aquellas garantizan una presentación uniforme que reduce la fricción cognitiva al momento de leer código escrito por distintos integrantes del equipo.**

La estandarización no busca limitar la creatividad del programador, sino reducir las decisiones innecesarias y facilitar la colaboración.

### Documentación del código

La documentación constituye un recurso fundamental para preservar el conocimiento asociado a un sistema. Sin embargo, documentar no significa agregar comentarios excesivos a cada línea de código. Un código claro y bien estructurado debe explicar, en gran medida, qué está haciendo.

Los comentarios son especialmente útiles para explicar aspectos que el código no puede expresar fácilmente, como:

- Decisiones de diseño.
- Restricciones técnicas.
- Reglas de negocio complejas.
- Motivos de una solución determinada.
- Consecuencias o riesgos de modificar un componente.

Además de los comentarios internos, la documentación puede incluir manuales técnicos, documentación de APIs, diagramas y archivos que expliquen cómo instalar, configurar y utilizar un proyecto.

Una documentación adecuada facilita la incorporación de nuevos integrantes al equipo y reduce la dependencia del conocimiento individual. Es importante mantenerla actualizada, ya que una documentación obsoleta puede resultar más perjudicial que la ausencia de documentación.

### Tipos y niveles de documentación

La documentación de software puede clasificarse en distintos niveles según su audiencia y propósito.

La documentación en línea, compuesta por comentarios estructurados dentro del propio código fuente —como los docstrings en Python o los bloques Javadoc en Java—, describe el comportamiento, los parámetros y los valores de retorno de funciones y clases específicas. Un segundo nivel corresponde a la documentación de proyecto, típicamente materializada en un archivo README, que ofrece una visión general sobre la instalación, configuración y uso del sistema.



---

A un nivel más alto se encuentra la documentación de arquitectura, que describe las decisiones de diseño estructural del sistema y, en muchos equipos, se registra mediante Registros de Decisiones de Arquitectura (ADR, por sus siglas en inglés), documentos breves que capturan el contexto, las alternativas consideradas y la justificación de decisiones técnicas relevantes.

Finalmente, en sistemas que exponen interfaces de programación, la documentación de API —frecuentemente generada mediante especificaciones como OpenAPI o Swagger— resulta indispensable para que otros equipos o servicios puedan integrarse correctamente.

### Principios para una documentación efectiva

Una recomendación ampliamente compartida en la literatura de ingeniería de software sostiene que los comentarios y la documentación deben priorizar explicar el «por qué» de una decisión —motivaciones, restricciones, alternativas descartadas— por sobre el «qué» hace el código, dado que este último debería resultar evidente a partir de nombres y estructuras claras.

Duplicar en comentarios información que ya es evidente en el código no solo resulta redundante, sino que introduce un riesgo adicional: la documentación puede desactualizarse con el tiempo si no se mantiene sincronizada con los cambios en la implementación, generando información engañosa que resulta peor que la ausencia total de documentación.

Por este motivo, muchos equipos priorizan la generación automática de documentación a partir del propio código fuente, mediante herramientas como Sphinx, JSDoc o Doxygen, de forma que la documentación evolucione junto con la base de código y se reduzca el riesgo de divergencia entre ambas.

La documentación, entendida de esta manera, opera como un puente entre el código y las personas que deben comprenderlo sin haberlo escrito, cumpliendo una función especialmente relevante durante los procesos de incorporación de nuevos integrantes al equipo y durante la revisión de cambios que afectan a componentes poco familiares para quien revisa.

### Modularidad, reutilización y mantenibilidad

La **modularidad** consiste en **dividir un sistema en componentes o módulos con responsabilidades claramente definidas**.

Cada módulo debe concentrarse en una parte específica del problema y mantener una relación controlada con los demás componentes.

Dos conceptos resultan centrales para evaluar la calidad de una descomposición modular: el acoplamiento, que mide el grado de dependencia entre distintos módulos,

y la cohesión, que mide el grado en que los elementos internos de un mismo módulo están relacionados funcionalmente entre sí.

El objetivo deseable en el diseño de software es lograr bajo acoplamiento entre módulos y alta cohesión dentro de cada uno de ellos. Un sistema con estas características resulta más fácil de comprender de manera incremental, permite realizar cambios localizados sin generar efectos colaterales inesperados en otras partes del sistema, y facilita la asignación de responsabilidades entre distintos equipos de desarrollo.

Esta organización permite reducir la complejidad y facilita el desarrollo, las pruebas y el mantenimiento. Un sistema modular también favorece la reutilización, ya que determinados componentes pueden utilizarse en diferentes partes de una aplicación o incluso en otros proyectos.

### Principios SOLID

En el contexto de la programación orientada a objetos, los principios conocidos por el acrónimo SOLID constituyen una guía influyente para lograr diseños modulares y mantenibles.



Estos principios son:

- **Responsabilidad única**, según el cual una clase debería tener una sola razón para cambiar;
- **Abierto/cerrado**, que propone que las entidades de software deben estar abiertas a extensión, pero cerradas a modificación;
- **Sustitución de Liskov**, que exige que las subclases puedan reemplazar a sus clases base sin alterar el comportamiento correcto del programa;
- **Segregación de interfaces**, que recomienda dividir interfaces amplias en otras más específicas y cohesivas;
- **Inversión de dependencias**, que promueve depender de abstracciones en lugar de implementaciones concretas.

---

Si bien estos principios surgieron específicamente en el contexto de la programación orientada a objetos, sus ideas subyacentes —bajo acoplamiento, responsabilidades bien delimitadas, dependencia de abstracciones— resultan aplicables, con adaptaciones, a otros paradigmas de programación, incluyendo enfoques funcionales y arquitecturas basadas en microservicios.

### Reutilización

La **reutilización** permite **aprovechar soluciones existentes en lugar de desarrollar nuevamente funcionalidades similares**, evitar la duplicación de esfuerzos y concentrar el mantenimiento de una determinada funcionalidad en un único lugar.

Sin embargo, reutilizar no significa crear componentes excesivamente genéricos. Un buen diseño debe buscar un equilibrio entre la flexibilidad y la simplicidad.

### Mantenibilidad

Por su parte, la **mantenibilidad** representa la **capacidad de un sistema para ser comprendido, corregido, adaptado y mejorado**.

Un software mantenible posee una estructura clara y permite realizar modificaciones sin afectar innecesariamente otras partes.

Los principios de diseño, la separación de responsabilidades y una adecuada arquitectura contribuyen directamente a mejorar estas características.

La mantenibilidad, por su parte, es reconocida como un atributo de calidad de software en normas internacionales como la familia ISO/IEC 25010, que la describe en términos de la facilidad con la que un sistema puede ser modificado para corregir defectos, mejorar su rendimiento o adaptarse a nuevos requisitos.

La modularidad y la reutilización, en este sentido, no son fines en sí mismos, sino medios orientados a maximizar la mantenibilidad del software a lo largo de su ciclo de vida.

Un sistema modular, cohesivo y con bajo acoplamiento constituye, además, una condición favorable para la gestión ordenada de dependencias externas, tema que se desarrolla en el capítulo siguiente, dado que los límites claros entre módulos facilitan aislar el impacto de actualizar o reemplazar una librería determinada.

### Gestión de dependencias y paquetes

Los proyectos de software modernos suelen utilizar bibliotecas, frameworks y componentes desarrollados por terceros. Se presentan los siguientes problemas,

|                                 |                                                                       |
|---------------------------------|-----------------------------------------------------------------------|
| <b>Conflictos de versión</b>    | conocido como <i>dependencyhell</i>                                   |
| <b>Dependencias Transitivas</b> | librerías que traen otras librerías sin que el desarrollador lo note. |
| <b>Abandono del Proyecto</b>    | una librería externa queda sin mantenimiento                          |
| <b>Riesgos de Seguridad</b>     | uso de dependencias con vulnerabilidades conocidas.                   |

Estos recursos permiten acelerar el desarrollo, pero también generan una serie de dependencias que deben ser gestionadas adecuadamente. Se puede distinguir:

|                                                  |                                                                                                                                                                                                                                        |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>¿Qué son las dependencias de software?</b>    | Las dependencias de software son componentes o soluciones de IT necesarias para que una aplicación funcione correctamente. Pueden incluir bibliotecas, marcos u otro software.                                                         |
| <b>¿Qué son las dependencias en un software?</b> | <ul style="list-style-type: none"> <li>Las dependencias en un software se refieren a elementos que son esenciales para el funcionamiento de otro software. Sin ellos, una aplicación puede ejecutarse de manera incorrecta.</li> </ul> |
| <b>¿Qué es un software dependiente?</b>          | Un dependiente de software implica que el software depende de componentes o código externos para funcionar, como bibliotecas o marcos.                                                                                                 |

- Una aplicación web creada con un framework como React. El uso de React es una dependencia; sin él, la aplicación no funcionará.
- Un script de Python que realiza operaciones matemáticas complejas con la biblioteca NumPy. [NumPy](#) ofrece capacidades de cálculo optimizadas. Los desarrolladores no necesitan crear estos componentes desde cero. En su lugar, incluyen la dependencia y utilizan su funcionalidad en su aplicación.

- 
- Las dependencias permiten a los desarrolladores centrarse en las partes únicas de su aplicación en lugar de reinventar funcionalidades estándar y comunes.
  - Facilitan la reutilización del código y aceleran los ciclos de desarrollo y la estandarización.
  - Sin embargo, el uso de dependencias introduce una cadena de componentes interconectados que deben gestionarse.

La **gestión de dependencias** consiste en identificar, incorporar, actualizar y controlar las bibliotecas necesarias para el funcionamiento de un proyecto.

Es el conjunto de prácticas, herramientas y procesos que permiten controlar, instalar, actualizar y mantener las librerías, frameworks o módulos externos que un proyecto de software necesita para funcionar de tal forma de asegurar **que** estas piezas externas sean compatibles, seguras y fáciles de mantener.

### Conceptos Claves

- **Dependencia**

Es un componente externo (librería, framework, módulo, paquete) que el software necesita.

Ejemplo: un proyecto en Python puede depender de numpy o pandas.

- **Archivo de definición de dependencias**

Documento donde se declara qué versiones de librerías usa el proyecto.

Ejemplo en Node.js → package.json

Ejemplo en Maven → pom.xml

- **Resolución de versiones**

Cuando dos librerías requieren distintas versiones de la misma dependencia, el gestor debe resolver el conflicto (a veces automáticamente, otras manualmente).

### Ejemplos:

1. **Bibliotecas y marcos:** casos de la primera son JavaScript (por ejemplo, React o jQuery). Y marcos como .NET, que proporcionan código reutilizable y simplifican el desarrollo.
2. **Sistemas de Gestión de Bases de Datos:** las aplicaciones que necesitan almacenar o recuperar información dependen de bases específicas, como MySQL o MongoDB, para ejecutar sus funciones.
3. **API (Interfaces de Programación de Aplicaciones):** actúan como puentes, conectando sistemas de software. Por ejemplo, una aplicación que muestra mapas puede depender de la API de Google Maps para proporcionar una geolocalización precisa.
4. **Requisitos del sistema operativo:** algunos programas necesitan versiones o plataformas determinadas del Sistema Operativo (Windows, macOS, Linux) para

funcionar eficazmente. Sin el entorno correcto, el software falla o se ejecuta de forma subóptima.

5. **Marcos o Framework:** Base estructural que proporciona una plantilla para crear aplicaciones.

### Tipos de Gestión de Dependencias

| Tipo de Dependencia        | Concepto                                                                                                                                                                                           | Ejemplo                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dependencias Directas      | <p>Las dependencias directas son aquellas en las que el software confía directamente.</p> <p>Suelen ser las más críticas, ya que resultan fundamentales para el funcionamiento de la solución.</p> | <p>si se está ejecutando una aplicación Python, el propio intérprete del mismo es una dependencia directa.</p>                                                                                                                                                                                                                                                                                                                                                           |
| Dependencias Transitivas   | <p>Las dependencias transitivas son componentes secundarios requeridos por las dependencias directas.</p>                                                                                          | <p>El paquete de software A necesita que también se instalen los paquetes B y C para que funcione correctamente</p> <p>Los paquetes de dependencias son transitivos, lo que significa que cuando un paquete A necesita el paquete B, y el paquete B necesita el paquete C, significa que el paquete A también necesita el paquete C, por lo que a veces se termina instalando un montón de paquetes extras a pesar de que simplemente se quería una sola aplicación.</p> |
| Dependencias No Utilizadas | <p>Las dependencias no utilizadas se asemejan a una herramienta extra que compras “por si acaso”, pero que nunca llegas usar.</p>                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Dependencias Opcionales    | <p>Las dependencias opcionales proporcionan funcionalidades adicionales.</p>                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

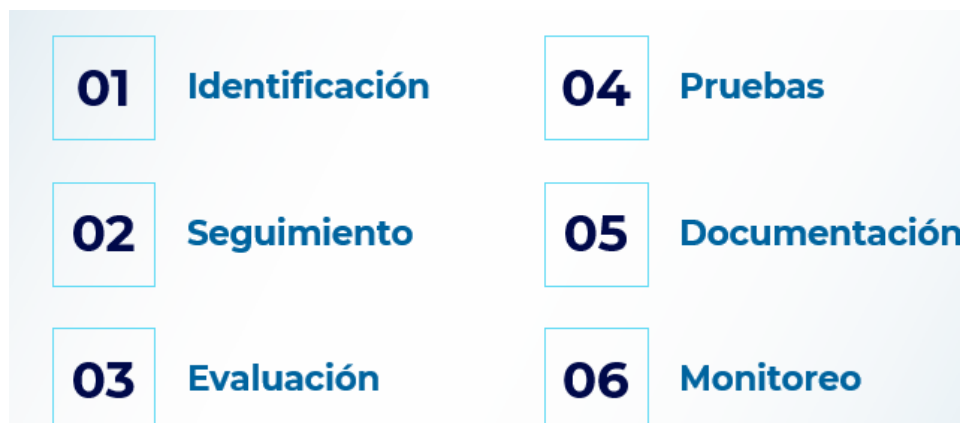


|  |                                                                                   |  |
|--|-----------------------------------------------------------------------------------|--|
|  | les, pero que no resultan críticas para el funcionamiento principal del software. |  |
|--|-----------------------------------------------------------------------------------|--|

### Buenas Prácticas de la Gestión de Dependencias

1. **Adopta un proceso de revisión regular:** establece un calendario para revisar periódicamente las dependencias de modo de garantizar que todos los componentes se encuentren actualizados.
2. **Prioriza las actualizaciones de seguridad:** las vulnerabilidades son comunes en las dependencias obsoletas, así que debes abordarlas rápidamente.
3. **Utiliza herramientas centralizadas para el seguimiento:** para mantener registros precisos de cada dependencia, lo que facilita el rastreo y la gestión.
4. **Documenta todo:** obtiene un panorama completo de la red de relaciones que facilitará también la cuestión para los nuevos miembros del equipo.
5. **Aprovecha la automatización:** las tareas rutinarias, como las comprobaciones de versiones y las pruebas de compatibilidad, son posible de automatizarse para ahorrar tiempo y reducir los costos.

### Como gestionar dependencias



**Identificación:** Identificar todos los componentes del entorno, lo cual incluye las directas y las transitivas que pueden no ser tan visibles.

**Seguimiento:** Hacer un seguimiento de cada componente, que implica anotar las versiones actuales, los requisitos de compatibilidad y cualquier actualización anterior para evitar problemas relacionados con desajustes de las versiones y componentes obsoletos.

**Evaluación:** Evaluar la relevancia y necesidad de cada componente. Las dependencias no utilizadas son particularmente riesgosas, ya que agregan desorden innecesario y vulnerabilidades de seguridad.

---

**Pruebas:** Garantizan que las dependencias funcionen bien con otro software, ayudan a identificar problemas de compatibilidad y reduce el riesgo de fallos o caídas de rendimiento.

**Documentación:** Consiste en documentar todas las dependencias y sus relaciones. Además, ayuda a los equipos a comprender la red y asiste en la capacitación de los nuevos miembros del equipo.

**Monitoreo:** Consiste en documentar todas las dependencias y sus relaciones. Además, ayuda a los equipos a comprender la red y asiste en la capacitación de los nuevos miembros del equipo.

### ¿Cómo gestionar las dependencias?

- Si bien las herramientas automatizadas son útiles, la inspección manual del código base sigue siendo esencial.
- Revisar las declaraciones de importación, las llamadas a bibliotecas y la documentación del proyecto puede proporcionar información valiosa sobre las dependencias de las que depende la aplicación y en qué contexto.
- Esto es especialmente importante para identificar dependencias innecesarias que se puedan eliminar y simplificar la implementación. También ayuda, si se utiliza un archivo de manifiesto, a ver qué dependencias se utilizan realmente.
- Por ejemplo, en Node, para usar una dependencia en un componente, es necesario importarla explícitamente. Inspeccionar el código permitirá verificar esto, ya que los archivos de manifiesto pueden incluir dependencias que no se utilizan al inspeccionar manualmente el código.

### Gestores de Paquetes

Un **PAQUETE** es un conjunto organizado de archivos y directorios que agrupa código fuente, recursos y metadatos relacionados con una funcionalidad específica, que incluyen todos los archivos necesarios para componer un software.

Es decir, **cajas que contienen herramientas y utilidades** que se puede añadir al proyecto de software, y que facilitan la construcción de programas y aplicaciones.

Dentro de un paquete, **es común encontrar uno o más bibliotecas**. La diferencia respecto a una librería es que vienen “paquetizados” (de ahí su nombre) para que sea más sencillo para ti incluirlos en tu trabajo.

Los paquetes pueden ser:

- Internas
- Open Source
- Terceros

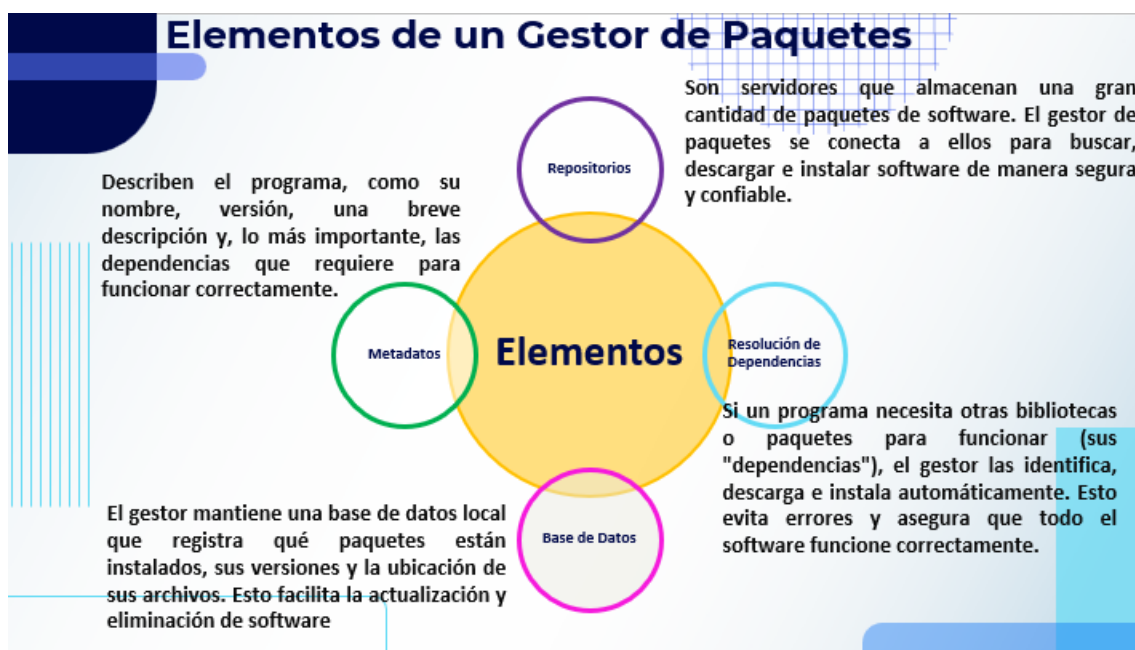
Un gestor de paquetes es una herramienta de software que automatiza la instalación, actualización, configuración y eliminación de programas informáticos o bibliotecas, llamados "paquetes".

#### Funciones de los Gestores de Paquetes:

- Son herramientas esenciales para gestionar dependencias.
- Automatizan la instalación, actualización y eliminación de paquetes y sus dependencias.
- También resuelven conflictos de versiones y cuentan con un repositorio centralizado para las dependencias.

#### Ventajas de usar gestores de paquetes

- **Automatización:** Elimina el tedioso proceso manual de descargar, instalar y configurar software, ahorrando tiempo y esfuerzo.
- **Resolución de dependencias:** Se encarga de las dependencias de manera automática, garantizando que el software funcione sin conflictos.
- **Actualizaciones sencillas:** Proporciona un método centralizado para mantener todo el software del sistema actualizado con los últimos parches y mejoras.
- **Seguridad y coherencia:** Los paquetes provienen de repositorios confiables, lo que reduce el riesgo de instalar software malicioso. Además, garantizan la consistencia al asegurar que se instalen las versiones correctas.
- **Eliminación limpia:** Permite desinstalar programas de forma completa y ordenada, eliminando todos los archivos y configuraciones relacionadas.



#### Gestores de Paquete

Los paquetes y los gestores de paquetes van unidos. Cada gestor de paquetes, usa su propio tipo de paquetes.

- Son específicos para cada lenguaje de programación.
- La mayoría de los lenguajes de programación, como C#, Python, Java o JavaScript o C++, por ejemplo, tienen uno o varios

---

formatos de paquetes y gestores de paquetes. En muchos casos, vienen incluidos con el entorno de desarrollo o el lenguaje mismo.

- Por último, los gestores de paquetes **suelen tener uno o varios ficheros de configuración**. Generalmente es un fichero de texto (en formato simple, toml, yaml, json...) que tiene un nombre específico.

### Ejemplo Gestor de Paquetes

Por ejemplo, Node.js utiliza npm (Gestor de Paquetes de Node) o yarn, Python utiliza pip, Java utiliza Maven o Gradle, y .NET utiliza NuGet. Estas herramientas utilizan un manifiesto o archivos de compilación (como package.json, requirements.txt o pom.xml) que especifican todas las dependencias directas del proyecto.

Al consultar este archivo, los desarrolladores pueden ver rápidamente las bibliotecas y los frameworks de los que depende su aplicación. Algunos IDE de código visualizan estas dependencias y enumeran todas las dependencias derivadas/indirectas que aportan las dependencias directas.

**C#** utiliza "NuGet" como su gestor de paquetes estándar. NuGet es una herramienta de línea de comandos y una extensión para Visual Studio que permite a los desarrolladores agregar, quitar y actualizar paquetes fácilmente en proyectos de **C#** y **.NET**. Por ejemplo:

```
1 nuget install NombrePaquete
2 nuget install NombrePaquete -Version Version
3 nuget uninstall NombrePaquete
```

En el pasado, **C#** utilizaba un archivo "packages.config" para gestionar las dependencias en proyectos más antiguos, pero con la introducción del enfoque de **PackageReference**, la gestión de dependencias se ha vuelto más sencilla y eficiente. En su lugar, ahora las dependencias se agregan directamente al archivo **.csproj**.

**Python** utiliza el gestor de paquetes "PIP" (pip installs packages). Con una simple línea de comandos, los desarrolladores pueden instalar, actualizar o eliminar paquetes. Por ejemplo:

```
1 pip install nombre_paquete
2 pip install nombre_paquete==versión
3 pip uninstall nombre_paquete
```

El fichero de configuración suele llamarse **requirements.txt**

En **C++** "Conan" se ha convertido en una popular herramienta de gestión de paquetes. Conan permite a los desarrolladores gestionar fácilmente bibliotecas y sus dependencias, lo que facilita el proceso de compilación y enlazado de proyectos en **C++**. Ejemplos de uso:

```
1 conan install NombrePaquete/Version
2 conan install NombrePaquete/Version@user/channel
3 conan remove NombrePaquete/Version
```

El fichero de configuración suele llamarse `conanfile.txt` o `conanfile.py`

#### Para concluir:

Entre los beneficios de una adecuada gestión de dependencias se encuentran:

- Facilitar la reproducción del entorno de desarrollo.
- Evitar incompatibilidades entre versiones.
- Simplificar la instalación del proyecto.
- Controlar actualizaciones y vulnerabilidades.
- Mantener un registro de los componentes externos utilizados.

Es recomendable evitar dependencias innecesarias y mantener actualizadas aquellas que sean fundamentales para el proyecto. Cada nueva dependencia introduce beneficios, pero también posibles riesgos y responsabilidades de mantenimiento. Por ello, la gestión de dependencias forma parte de la calidad y seguridad del software.

#### Gestores de paquetes por ecosistema

Prácticamente todos los lenguajes de programación modernos cuentan con un ecosistema de gestión de paquetes que facilita la incorporación, actualización y distribución de librerías externas. Entre los más utilizados se encuentran npm, yarn y pnpm en el ecosistema de JavaScript y TypeScript; pip y Poetry en Python; Maven y Gradle en Java; Cargo en Rust; y NuGet en el entorno .NET. Estas herramientas automatizan la resolución de dependencias transitivas, es decir, aquellas librerías de las que dependen, a su vez, las dependencias directas del proyecto.

El uso adecuado de estas herramientas incluye la separación explícita entre dependencias necesarias en tiempo de ejecución y aquellas utilizadas únicamente durante el desarrollo o las pruebas, evitando que estas últimas incrementen innecesariamente el tamaño de los artefactos desplegados en producción.

### Control de versiones

El **control de versiones** es una práctica fundamental en el desarrollo de software.

Permite registrar los cambios realizados sobre el código fuente a lo largo del tiempo y recuperar versiones anteriores cuando sea necesario, registrar de manera sistemática la

---

evolución histórica de un proyecto de software, posibilitando identificar quién realizó un cambio determinado, cuándo se realizó y por qué motivo, además de permitir revertir modificaciones problemáticas.

Los primeros sistemas de este tipo, como CVS y posteriormente Subversion (SVN), utilizaban un modelo centralizado, en el que existía un único repositorio remoto contra el cual todos los desarrolladores sincronizaban sus cambios.

Git, desarrollado originalmente por Linus Torvalds en 2005 para el mantenimiento del núcleo de Linux, introdujo un modelo distribuido en el que cada desarrollador posee una copia completa del historial del proyecto, lo que permite trabajar sin conexión permanente al repositorio central y ha consolidado a Git como el estándar de facto en la industria del software.

Los sistemas de control de versiones, como Git, facilitan el trabajo individual y colaborativo. Cada modificación puede registrarse mediante un **commit**, que representa una unidad de cambio con un historial asociado.

Entre las principales ventajas del control de versiones se encuentran:

- Mantener un historial de cambios.
- Recuperar versiones anteriores.
- Desarrollar funcionalidades en paralelo.
- Facilitar la colaboración entre desarrolladores.
- Identificar quién realizó determinados cambios.
- Reducir el riesgo de pérdida de información.

Para aprovechar correctamente estas herramientas, es conveniente realizar commits pequeños y coherentes, acompañados de mensajes claros que describan el cambio realizado.

**El control de versiones no debe considerarse simplemente un mecanismo de respaldo, sino una herramienta central para organizar el proceso de desarrollo.**

### Buenas prácticas de commits

Una práctica ampliamente recomendada consiste en realizar commits atómicos, es decir, cambios que representen una unidad lógica completa y coherente, evitando mezclar en un mismo commit modificaciones no relacionadas entre sí. Esta práctica facilita tanto la revisión posterior del historial como la posibilidad de revertir selectivamente cambios problemáticos sin afectar otras modificaciones.

Asimismo, convenciones como Conventional Commits proponen estructurar los mensajes de commit mediante prefijos estandarizados —como feat, fix, docs o refactor— que indican la naturaleza del cambio realizado, permitiendo generar automáticamente registros de cambios (changelogs) y facilitando la comprensión del historial del proyecto por parte de todo el equipo.

---

Sobre esta base de control de versiones se construyen las estrategias de ramificación que se analizan en el capítulo siguiente, las cuales definen de qué manera se organiza el trabajo simultáneo de distintos desarrolladores sobre un mismo repositorio.

### Versionado semántico y reproducibilidad

El versionado semántico (SemVer) constituye una convención ampliamente adoptada para comunicar de manera predecible el impacto de los cambios en una librería, mediante un esquema de tres números —versión mayor, menor y de parche— donde los cambios incompatibles incrementan la versión mayor, las nuevas funcionalidades compatibles incrementan la versión menor, y las correcciones de errores incrementan la versión de parche.

Para garantizar la reproducibilidad de las compilaciones, resulta fundamental el uso de archivos de bloqueo (lock files), como `package-lock.json` o `poetry.lock`, que fijan de manera exacta las versiones instaladas de todas las dependencias, incluidas las transitivas, evitando que distintas instalaciones del mismo proyecto —en distintas máquinas o momentos— resulten en configuraciones de dependencias diferentes.

### Riesgos de seguridad y buenas prácticas

La incorporación de dependencias externas introduce, junto con sus beneficios, riesgos de seguridad asociados a vulnerabilidades conocidas en dichas librerías, así como riesgos de mantenimiento derivados de proyectos externos que pueden quedar abandonados por sus autores. Por este motivo, se recomienda evaluar criterios como la actividad de mantenimiento, la licencia de uso y el historial de vulnerabilidades antes de incorporar una nueva dependencia a un proyecto.

Herramientas automatizadas como Dependabot o Snyk permiten escanear periódicamente las dependencias de un proyecto en busca de vulnerabilidades conocidas y sugerir o aplicar automáticamente actualizaciones correctivas, integrando la gestión de seguridad de dependencias dentro del flujo habitual de desarrollo.

La gestión responsable de dependencias externas constituye, en definitiva, una extensión natural de la mantenibilidad del sistema hacia su cadena de suministro de software, un aspecto cuya relevancia se ha incrementado notablemente en la última década a medida que los proyectos modernos incorporan un número creciente de librerías de terceros.

### Estrategias de ramificación y flujos de trabajo

Las **ramas o branches** permiten desarrollar **cambios de manera aislada sin modificar directamente la versión principal del software**.

A partir de esta funcionalidad se han desarrollado diferentes estrategias para organizar el trabajo de los equipos.



---

## Git Flow

Git Flow es un modelo de ramificación propuesto por Vincent Driessen en 2010, orientado a proyectos con ciclos de lanzamiento planificados y versiones formales. El modelo define ramas de larga duración, como `main`, que representa el código en producción, y `develop`, utilizada para integrar el trabajo en curso; junto con ramas de corta duración específicas, como `feature` para el desarrollo de nuevas funcionalidades, `release` para la preparación de versiones y `hotfix` para correcciones urgentes sobre producción.

Para concluir, **Git Flow** es una estrategia basada en la utilización de ramas con propósitos específicos. Generalmente incluye una rama principal para las versiones estables y otra destinada a la integración del desarrollo, además de ramas temporales para nuevas funcionalidades, correcciones y versiones. Este modelo ofrece una estructura clara para proyectos que requieren ciclos de desarrollo y publicación bien definidos.

Sin embargo, puede resultar complejo para equipos pequeños o para proyectos que necesitan realizar entregas frecuentes. Si bien Git Flow ofrece un control riguroso sobre el proceso de versionado, su complejidad estructural puede generar dificultades en equipos con integración muy frecuente, dado que el tiempo prolongado de vida de las ramas incrementa la probabilidad de conflictos de fusión extensos y complejos de resolver.

## Trunk-Based Development

El **desarrollo basado en trunk** propone mantener los cambios integrados con frecuencia en una rama principal compartida. Las ramas, cuando se utilizan, tienen una duración breve, es decir que todos los integrantes del equipo integran sus cambios de manera frecuente —habitualmente varias veces al día— directamente sobre una única rama principal, utilizando ramas de vida muy corta, de horas o, a lo sumo, uno o dos días de duración.

Este enfoque busca reducir los problemas derivados de integraciones tardías y favorece la entrega continua. Para aplicarlo adecuadamente es necesario contar con pruebas automatizadas y mecanismos que permitan integrar cambios de manera segura.

La elección entre una estrategia u otra depende de las características del proyecto, del tamaño del equipo y de la frecuencia de las entregas. No existe un único flujo de trabajo adecuado para todas las situaciones.

Para gestionar funcionalidades incompletas sin bloquear la integración continua, este enfoque se apoya fuertemente en el uso de `feature flags` o banderas de funcionalidad, mecanismos que permiten desactivar en tiempo de ejecución código que aún no está listo para ser expuesto a los usuarios finales. Esta estrategia resulta particularmente adecuada para organizaciones que practican integración y despliegue continuos, dado

---

que reduce significativamente la magnitud de los conflictos de fusión al minimizar el tiempo durante el cual el código diverge del tronco principal.

### Modelos intermedios y criterios de selección

Entre ambos extremos existen modelos intermedios, como GitHub Flow y GitLab Flow, que simplifican Git Flow manteniendo una única rama principal desplegable, junto con ramas de funcionalidad de corta duración que se integran mediante pull requests, incorporando en algunos casos ramas adicionales para representar distintos entornos de despliegue, como staging o producción.

La elección de una estrategia de ramificación no responde a una única solución universalmente óptima, sino que depende de factores como el tamaño del equipo, la frecuencia de despliegue deseada, la madurez de la automatización de pruebas e integración continua, y los requisitos regulatorios o de trazabilidad de versiones propios de cada organización.

### Pull Requests y Merge Requests

Las prácticas de **Pull Request (PR)** y **Merge Request (MR)** permiten **revisar y discutir los cambios antes de incorporarlos a una rama compartida del proyecto**. Un desarrollador realiza su trabajo en una rama y posteriormente solicita que sus modificaciones sean revisadas. Los demás integrantes pueden analizar el código, realizar comentarios, solicitar modificaciones y finalmente aprobar la integración.

Un pull request (denominado merge request en plataformas como GitLab) es una solicitud formal para integrar los cambios realizados en una rama secundaria hacia la rama principal de un repositorio, que habilita un espacio estructurado de revisión, discusión y validación automatizada antes de que dicha integración se concrete.

Esta práctica se ha convertido en el mecanismo estándar de colaboración en proyectos de software modernos, independientemente de la estrategia de ramificación adoptada.

Más allá de su función técnica de integración de código, los pull requests cumplen un rol documental relevante, dado que registran de manera permanente el contexto, las alternativas consideradas y las discusiones asociadas a cada cambio significativo introducido en el sistema.

Entre los beneficios de este mecanismo se encuentran:

- Mejorar la calidad del código.
- Detectar errores antes de la integración.
- Compartir conocimientos entre los miembros del equipo.
- Mantener un registro de las decisiones técnicas.
- Favorecer la aplicación de estándares.

---

### Buenas prácticas en su elaboración

Una buena Pull Request debe ser relativamente pequeña, estar enfocada en un objetivo concreto y proporcionar información suficiente para que otros desarrolladores puedan comprender los cambios.

Es decir, se recomienda que los pull requests sean de tamaño reducido y estén enfocados en un único propósito, dado que los cambios extensos resultan considerablemente más difíciles de revisar con el nivel de atención adecuado, incrementando la probabilidad de que errores relevantes pasen inadvertidos durante la revisión. Asimismo, cada pull request debería incluir una descripción clara del cambio realizado, su motivación, y —cuando corresponda— instrucciones para probarlo manualmente.

Previo a solicitar la revisión, resulta conveniente verificar que el código cumpla con una serie de criterios mínimos: que las pruebas automatizadas se ejecuten correctamente, que las herramientas de análisis estático no reporten incidencias, y que no se incluya código muerto, comentado o de depuración que debería haberse eliminado antes de la integración.

### Integración con la automatización de calidad

En la mayoría de los flujos de trabajo modernos, la apertura de un pull request dispara automáticamente un conjunto de verificaciones mediante herramientas de integración continua, que incluyen la compilación del proyecto, la ejecución de la batería completa de pruebas automatizadas y el análisis estático del código. La política habitual consiste en impedir la fusión del pull request hasta que todas estas verificaciones se completen exitosamente, lo que constituye una barrera automatizada de calidad previa a cualquier revisión humana.

El pull request constituye, de este modo, el punto de encuentro entre la revisión humana del código —desarrollada en profundidad en el capítulo dedicado a code review— y la verificación automatizada de calidad propia de la integración continua.

## Trabajo colaborativo y resolución de conflictos

El desarrollo de software es, habitualmente, una actividad colectiva. Diferentes integrantes pueden trabajar simultáneamente sobre un mismo proyecto, lo que requiere mecanismos de coordinación y comunicación.

Uno de los problemas frecuentes es la aparición de **conflictos de integración**, que ocurren cuando distintas personas modifican partes incompatibles del mismo código.

Los sistemas de control de versiones pueden identificar estos conflictos, pero su resolución requiere analizar cuidadosamente las modificaciones realizadas.

Para reducir estos inconvenientes se recomienda:

- Realizar integraciones frecuentes.

- 
- Mantener las ramas actualizadas.
  - Dividir el trabajo en tareas claramente definidas.
  - Evitar cambios excesivamente grandes.
  - Mantener una comunicación constante dentro del equipo.

La resolución de conflictos no debe ser considerada únicamente como un problema técnico. También requiere comprender la intención de los cambios y dialogar con los demás integrantes cuando sea necesario.

### **Coordinación en equipos de desarrollo**

El desarrollo de software en equipo requiere mecanismos explícitos de coordinación que permitan a distintas personas trabajar simultáneamente sobre una misma base de código sin generar interferencias significativas entre sí. La comunicación temprana sobre cambios que afectan a módulos compartidos, la integración frecuente de trabajo en curso y el uso de tableros de tareas o sistemas de seguimiento de incidencias contribuyen a reducir la probabilidad de que dos personas modifiquen simultáneamente, y de forma incompatible, una misma porción del sistema.

En equipos distribuidos geográficamente, estas prácticas cobran una relevancia adicional, dado que las diferencias de husos horarios y la comunicación predominantemente asincrónica exigen una documentación más explícita de las decisiones y del estado de avance de cada tarea.

### **Conflictos de fusión: causas y resolución**

Un conflicto de fusión (merge conflict) surge cuando dos ramas de desarrollo modifican de manera incompatible una misma porción de un archivo, y el sistema de control de versiones no puede determinar automáticamente cómo combinar ambos cambios. Cuanto mayor es el tiempo que una rama permanece sin integrarse con la rama principal, mayor es la probabilidad de que se acumulen conflictos de fusión extensos y difíciles de resolver correctamente.

La resolución adecuada de un conflicto requiere comprender la intención de ambos cambios en disputa, antes de simplemente elegir una de las dos versiones o combinarlas de manera mecánica. Se recomienda, además, ejecutar las pruebas automatizadas del proyecto inmediatamente después de resolver un conflicto, dado que una fusión sintácticamente correcta puede, sin embargo, introducir errores semánticos que solo se manifiestan en tiempo de ejecución.

### **Prácticas colaborativas complementarias**

Prácticas como la programación en pareja (pair programming) y la programación en grupo (mob programming), en las que dos o más desarrolladores trabajan simultáneamente sobre el mismo código, comparten contexto en tiempo real, lo que reduce la probabilidad de conflictos posteriores y favorece la transferencia de conocimiento dentro del equipo, particularmente en tareas complejas o de alto riesgo.

---

Estas prácticas colaborativas se articulan, en el flujo de trabajo habitual de un equipo de desarrollo, con el mecanismo formal de integración de cambios que constituyen los pull requests.

## Refactorización continua

La **refactorización** consiste en modificar la estructura interna del código sin alterar su comportamiento externo.

Se trata, por lo tanto, de una actividad distinta tanto de la corrección de errores como de la incorporación de nuevas funcionalidades, aunque frecuentemente se realiza de manera intercalada con ambas.

Su propósito es mejorar aspectos como la legibilidad, la organización y la mantenibilidad. Con el paso del tiempo, un proyecto puede acumular duplicaciones, funciones demasiado extensas y estructuras difíciles de comprender. La refactorización permite corregir estos problemas progresivamente.

Algunos ejemplos de refactorización incluyen:

- Dividir funciones extensas.
- Mejorar nombres de variables y métodos.
- Eliminar código duplicado.
- Simplificar estructuras condicionales.
- Extraer componentes o módulos.
- Reducir el acoplamiento entre componentes.

La refactorización continua evita que los problemas de diseño se acumulen y se conviertan en una deuda técnica difícil de resolver. Es recomendable acompañarla con pruebas que permitan comprobar que el comportamiento del software se mantiene correctamente.

Una heurística difundida en la cultura ágil, conocida informalmente como la «regla del explorador» (Boy Scout Rule), sostiene que cada desarrollador debería procurar dejar el código en un estado ligeramente mejor de como lo encontró, incorporando pequeñas mejoras incrementales cada vez que trabaja sobre una determinada porción del sistema, en lugar de posponer la mejora estructural del código a proyectos de refactorización de gran escala que rara vez llegan a concretarse.

### Señales que motivan la refactorización

La literatura especializada identifica una serie de señales de diseño deficiente, conocidas como «malos olores de código» (code smells), que suelen motivar una intervención de refactorización. Entre las más frecuentes se encuentran las funciones excesivamente

---

extensas, la duplicación de lógica en distintas partes del sistema, las clases que concentran demasiadas responsabilidades no relacionadas entre sí, y el acoplamiento excesivo entre módulos que deberían ser independientes.

La detección temprana de estas señales, ya sea mediante inspección manual durante la revisión de código o mediante herramientas de análisis estático que calculan métricas de complejidad, permite intervenir antes de que el deterioro estructural del sistema alcance niveles que dificulten significativamente su evolución futura.

### **La necesidad de una red de seguridad de pruebas**

Dado que la refactorización debe preservar el comportamiento externo del sistema, su práctica segura depende críticamente de la existencia de una suite de pruebas automatizadas confiable, capaz de detectar rápidamente cualquier alteración no intencionada en el comportamiento del software. En ausencia de esta red de seguridad, la refactorización se convierte en una actividad de alto riesgo, dado que no existe manera sistemática de verificar que los cambios estructurales no hayan introducido regresiones.

La refactorización continua se articula, asimismo, con la revisión de código: muchas oportunidades de mejora estructural se identifican precisamente durante el proceso de revisión de un pull request, cuando una persona distinta del autor original observa el código con una perspectiva fresca.

### **Revisión de código (Code Review)**

La **revisión de código** es el proceso mediante el cual uno o más desarrolladores analizan las modificaciones realizadas por otro integrante del equipo.

Su objetivo no es encontrar culpables ni evaluar personalmente al programador. La revisión debe entenderse como una práctica de aprendizaje y mejora colectiva.

Durante una revisión pueden analizarse aspectos como:

- Corrección de la solución implementada.
- Claridad y legibilidad del código.
- Cumplimiento de estándares.
- Posibles errores o casos no contemplados.
- Seguridad y rendimiento.
- Existencia de pruebas adecuadas.

Una revisión efectiva debe centrarse en el código y no en la persona. Los comentarios deben ser respetuosos, específicos y orientados a mejorar la solución.

Además de detectar problemas, el Code Review favorece la transferencia de conocimientos. Los integrantes del equipo pueden conocer diferentes partes del sistema y aprender nuevas técnicas y enfoques.

---

### **Buenas prácticas para quienes revisan**

Se recomienda que la revisión de código se concentre principalmente en aspectos de lógica, diseño, legibilidad y cobertura de casos límite, delegando en herramientas automatizadas —linters y formateadores— la verificación de cuestiones puramente estilísticas. Asimismo, resulta fundamental que la retroalimentación brindada sea específica, constructiva y respetuosa, distinguiendo claramente entre observaciones que constituyen un impedimento para aprobar el cambio y sugerencias opcionales de mejora.

La extensión de los cambios sometidos a revisión también incide directamente en la calidad de esta: revisiones que abarcan una cantidad excesiva de líneas modificadas tienden a recibir una atención superficial, disminuyendo la probabilidad de detectar problemas relevantes.

### **Buenas prácticas para quienes reciben la revisión**

Desde la perspectiva de quien recibe las observaciones, resulta importante interpretar los comentarios de revisión como una contribución orientada a mejorar el producto colectivo, y no como una crítica personal. Responder de manera explícita a cada observación recibida —ya sea aplicando el cambio sugerido o justificando fundadamente por qué se opta por no hacerlo— contribuye a mantener un proceso de revisión transparente y a preservar un registro útil de las decisiones de diseño adoptadas.

La revisión de código, en conjunto con las pruebas automatizadas y el análisis estático descritos en el capítulo final de esta monografía, conforma una de las principales barreras de contención frente a la introducción de defectos en el software antes de que estos alcancen los entornos de producción.

## **Introducción al aseguramiento de la calidad del software durante la implementación**

### **La calidad como proceso continuo**

A diferencia de una concepción tradicional que ubicaba el control de calidad exclusivamente como una fase final previa a la entrega del software, los enfoques contemporáneos de ingeniería de software conciben la calidad como una preocupación transversal, presente desde las primeras etapas de implementación de cada funcionalidad. Esta perspectiva se apoya en la evidencia de que el costo de corregir un defecto se incrementa considerablemente cuanto más tardíamente se detecta dentro del ciclo de vida del proyecto.

Bajo este enfoque, prácticas como las descritas previamente —código limpio, revisión de código, refactorización continua— se conciben como componentes integrales de la estrategia de calidad, en lugar de actividades aisladas de un proceso de control de calidad independiente.



El **aseguramiento de la calidad del software**, conocido como Software Quality Assurance (SQA), comprende las actividades orientadas a garantizar que el proceso y el producto de software cumplan con los niveles de calidad establecidos.

La calidad no debe verificarse únicamente al finalizar el desarrollo. Es necesario incorporarla durante todas las etapas del ciclo de vida, especialmente durante la implementación.

En esta etapa pueden aplicarse diferentes prácticas:

- Uso de estándares de codificación.
- Análisis estático del código.
- Revisiones de código.
- Pruebas unitarias.
- Pruebas de integración.
- Automatización de pruebas.
- Integración continua.
- Control y gestión de defectos.

El enfoque preventivo resulta especialmente importante. Detectar un problema durante la implementación suele ser menos costoso que descubrirlo cuando el sistema ya se encuentra en producción.

La calidad del software también puede analizarse desde diferentes atributos, entre ellos la confiabilidad, la seguridad, la eficiencia, la mantenibilidad y la compatibilidad. Estos aspectos deben ser considerados desde las primeras decisiones de diseño y desarrollo.

Por lo tanto, el aseguramiento de la calidad debe integrarse en las prácticas cotidianas de los equipos de desarrollo y no constituir una actividad separada que se realiza únicamente al finalizar un proyecto.

### **12.3 Integración y despliegue continuos**

La integración continua consiste en la práctica de integrar los cambios de código de manera frecuente a un repositorio compartido, ejecutando automáticamente en cada integración un conjunto de verificaciones que incluyen la compilación del proyecto y la ejecución de pruebas automatizadas. El despliegue continuo extiende esta práctica, automatizando también la publicación del software en entornos de prueba o producción una vez superadas dichas verificaciones.

Estas prácticas requieren, como condición necesaria, una suite de pruebas automatizadas confiable y suficientemente rápida, dado que su valor depende de la capacidad de detectar problemas en un plazo breve luego de introducidos, evitando que se acumulen en el sistema durante períodos prolongados.

---

## Métricas y análisis estático

Diversas herramientas de análisis estático, como SonarQube, permiten calcular métricas objetivas sobre la calidad del código, incluyendo la cobertura de pruebas, la complejidad ciclomática de las funciones y la presencia de vulnerabilidades de seguridad conocidas. Si bien estas métricas constituyen un insumo valioso para la toma de decisiones, resulta importante advertir que ninguna métrica aislada —incluida la cobertura de pruebas— garantiza por sí sola la ausencia de defectos, dado que es posible alcanzar una cobertura elevada con pruebas que verifican escenarios triviales o poco representativos del comportamiento real del sistema.

Por este motivo, muchos equipos complementan estas métricas cuantitativas con criterios cualitativos explícitos, formalizados en una «definición de terminado» (Definition of Done) compartida por todo el equipo, que especifica de manera concreta las condiciones que debe cumplir una tarea para considerarse efectivamente completada, incluyendo, típicamente, la revisión de código, la ejecución exitosa de pruebas y la actualización de la documentación correspondiente.

## Conclusión

Las buenas prácticas de programación representan un componente fundamental para el desarrollo de software de calidad. La implementación de un sistema no consiste únicamente en transformar requisitos en código funcional, sino también en construir una base tecnológica que pueda ser comprendida, mantenida y evolucionada a lo largo del tiempo.

El Código Limpio, las convenciones de codificación y una adecuada documentación favorecen la comprensión del software y reducen la complejidad. Del mismo modo, la modularidad y la reutilización permiten construir sistemas más flexibles y mantenibles.

El control de versiones y las estrategias de ramificación proporcionan mecanismos esenciales para organizar el trabajo individual y colaborativo. Prácticas como las Pull Requests, la revisión de código y la resolución adecuada de conflictos permiten mejorar la comunicación y detectar problemas antes de que se integren al producto.

Por otra parte, la refactorización continua permite preservar y mejorar la calidad interna del código, evitando la acumulación de deuda técnica. Estas prácticas, acompañadas por mecanismos de pruebas y aseguramiento de la calidad, contribuyen a que los equipos puedan desarrollar software de manera más confiable.

En conclusión, la calidad del software no es el resultado de una única herramienta o actividad, sino de la integración de múltiples prácticas durante todo el proceso de desarrollo. Adoptar buenas prácticas durante la implementación permite no solo producir sistemas que funcionen correctamente, sino también construir soluciones sostenibles, adaptables y preparadas para responder a las necesidades futuras.

---

## Estándares de Codificación

**Son un conjunto de reglas y directrices que un equipo de desarrollo acuerda seguir para escribir código de manera consistente y uniforme.**

**Objetivo:** Mejorar la calidad del software haciendo que el código sea más legible, mantenible y escalable.

- Los estándares de codificación son un conjunto de directrices y buenas prácticas que los desarrolladores siguen al escribir código.
- Estos estándares abarcan diversos aspectos, como las convenciones de nomenclatura, la organización del código, la sangría, los comentarios, la gestión de errores, etc.
- Considere los estándares de codificación como reglas, técnicas y buenas prácticas para desarrollar código más limpio, legible y eficiente, con un mínimo de errores.

Los estándares de codificación ayudan al equipo a crear código y software de alta calidad.

**Pero ¿qué significa código de alta calidad?**

**Un código de alta calidad garantiza que el software sea seguro y confiable para los programadores y sus usuarios.**

### Beneficios de aplicar estándares de Codificación

- **Mejora la legibilidad y claridad:** Un código consistente es más fácil de leer, entender y procesar para cualquier desarrollador, reduciendo la carga cognitiva.
- **Aumenta la mantenibilidad:** El mantenimiento y las modificaciones se vuelven menos complejos, ya que la uniformidad facilita la comprensión del código escrito por otros.
- **Facilita la colaboración:** En equipos grandes, los estándares garantizan que todo el código tenga una apariencia similar, independientemente de quién lo haya escrito. Esto reduce el tiempo que los desarrolladores nuevos tardan en integrarse a un proyecto.
- **Reduce los errores y defectos:** Al promover prácticas de programación sólidas, los estándares ayudan a prevenir errores comunes y fallas de seguridad, lo que se traduce en un software de mayor calidad.
- **Acelera el desarrollo:** Menos tiempo dedicado a resolver inconsistencias y a descifrar código complejo permite a los desarrolladores enfocarse en la creación de nuevas funcionalidades.

### Estándar de codificación abierto

Un "estándar de codificación abierto" se refiere a pautas de codificación accesibles al público, lo que permite la colaboración y las aportaciones de la comunidad.

---

Un ejemplo es "CERT", un estándar de codificación seguro que adopta la forma de una wiki web abierta y pública. Los estándares se crean mediante un esfuerzo colaborativo de las comunidades de desarrollo de software y seguridad.

Los estándares de codificación segura CERT, fueron desarrollados por el Instituto de Ingeniería de Software de la Universidad Carnegie Mellon.

Los estándares de codificación segura CERT proporcionan reglas y recomendaciones para varios lenguajes de programación, como C, C ++, Java y Python, así como para plataformas específicas, como Android e iOS.

Los estándares de codificación segura de CERT tienen como objetivo ayudar a los desarrolladores a evitar comportamientos indefinidos, errores y vulnerabilidades que pueden provocar violaciones de seguridad, pérdida de datos o fallas del sistema.

También proporcionan ejemplos, explicaciones y referencias para cada regla y recomendación.

### **Estándares de Codificación Cerrados**

Por el contrario, un "estándar de codificación cerrado" se refiere a las reglas de codificación que se aplican dentro de una empresa. Estas reglas no se comparten públicamente, lo que significa que solo los desarrolladores de esa organización pueden acceder a las directrices detalladas.

Un ejemplo de un estándar de codificación cerrado podría ser la guía de estilo interna de una empresa que dicta convenciones de nomenclatura precisas, prácticas de comentarios y patrones de codificación específicos exclusivos de la arquitectura de su sistema, que no revelan a los desarrolladores externos.

### **Buenas Prácticas para implementar estándares de codificación**

#### **UTILICE LINTERS Y FORMATEADORES**

**Los linters** (p. ej., ESLint para JavaScript, Flake8 para Python) y los formateadores (p. ej., Prettier, Black) detectan errores de estilo y ajustan automáticamente el formato del código para cumplir con las pautas de estilo predefinidas. Esto garantiza la coherencia en aspectos como la sangría, el espaciado y los saltos de línea.

Para evitar la tediosa tarea de ejecutar estas herramientas cada vez que realice un cambio, intégrealas directamente en su flujo de trabajo de desarrollo (IDE, proveedor de Git, etc.) y utilice ganchos de preconfirmación para revisar automáticamente el código antes de confirmarlo

#### **REVISIONES DE CÓDIGO AUTOMATIZADAS**

Estas herramientas agilizan las revisiones de código al analizarlo en busca de errores, vulnerabilidades de seguridad e infracciones de los estándares de codificación, lo que

---

proporciona una verificación preliminar de la calidad del código antes de una revisión manual.

Por ejemplo [GitHub Actions](#) permite aplicar estándares automáticamente durante las revisiones de código.

### **PROMOVER LAS REVISIONES DE CÓDIGO ENTRE PARES**

Las revisiones periódicas entre pares permiten a los desarrolladores verificar activamente el código de los demás en función de las pautas establecidas, lo que les permite identificar problemas a tiempo y realizar los ajustes necesarios.

Un consejo práctico que surgió de nuestras conversaciones fue asegurarse de que sus estándares de codificación aborden problemas reales:

### **INSTALAR POLÍTICAS DE CONTROL DE VERSIONES**

Su rama principal es una de las partes más importantes del negocio y debe protegerse a toda costa. Es importante aplicar los estándares de codificación mediante políticas de ramificación y fusión, estableciendo reglas que deben seguirse antes de fusionar el código con la base de código principal.

Por ejemplo, una política de ramificación podría requerir que los desarrolladores creen ramas de características para tareas específicas, garantizando así que solo se fusione el código completado, revisado y probado.

Las políticas de fusión pueden luego implementar controles, como requerir una revisión del código, aprobar pruebas automatizadas o confirmar el cumplimiento de las pautas de estilo antes de permitir la fusión.

### **Estándares de la industria**

- Python: [Guía de estilo PEP8 \(Propuesta de mejora de Python\) para código Python](#)
- Java: [Guía de estilo de Java de Google](#)
- Javascript: [Guía de estilo de JavaScript de Google](#)
- C#: [Convenciones de codificación de Microsoft C#](#)
- C/C++: [CERTIFICACIÓN SEI para C/C++](#)
- PHP: [Estándares de codificación PHP de WordPress](#)
- Objective-C: [Guía de codificación Objective-C de Apple para Cocoa](#)
- TypeScript: [Guía de estilo de Google TypeScript](#)

### **Ejemplo**

## 1. Convenciones de nomenclatura

Asigne nombres claros y significativos a sus variables, funciones y métodos para ayudar a otros y a usted mismo en el futuro a comprender rápidamente la intención y el comportamiento de su código.

He aquí un ejemplo de una convención de nomenclatura deficiente:

```
1 // Malo
2 let x = 10;
3 función fn(a) {
4     return a * 2;
5 }
```

Los nombres "x" y "fn" no describen nada sobre el código. La regla general es que si tienes que leer el código para entender su función, necesitas nombrar mejor tus variables y funciones. Aquí tienes una versión mejorada:

```
1 // Bien,
2 let itemCount = 10;
3 función doubleValue(valor) {
4     valor de retorno * 2;
5 }
```

Para mantener la coherencia en todo el código base, elige un estilo de nomenclatura y ciñete a él. Por ejemplo, si eliges camelCase, no cambies a snake\_case en el futuro.

## 2. Formato y sangría

La consistencia de la sangría, los saltos de línea y el formato son fundamentales para que el código sea más legible y comprensible. A continuación, se muestra un ejemplo que compara una buena práctica con una mala.

Lo malo:

```
1 | def calcular_total(precio, cantidad):total=precio*cantidad;devolver total
```

Este código es un desastre. Ni siquiera se ejecuta porque Python usa sangría para definir bloques de código.

Esto se ve mucho mejor y funcionaría sin problemas:

```
1 | def calculate_total(precio, cantidad):
2 |     total = precio * cantidad
3 |     return total
```

Instala un formateador de código (como Prettier para JavaScript o Black para Python) directamente en tu IDE para formatear automáticamente tu código. Además, configura ganchos de precommit para asegurar que todo el código nuevo se formatee antes de ser enviado.

### Convenciones de Codificación

**Las convenciones de codificación son pautas específicas de cada lenguaje de programación que ofrecen recomendaciones para mantener el código limpio.**

Estas recomendaciones se centran principalmente en el estilo de programación y las mejores prácticas de organización. Ayudan a mantener la coherencia, pero permiten cierta flexibilidad.

Diferentes lenguajes de programación y comunidades pueden tener sus convenciones específicas, pero aquí hay algunas convenciones de codificación comunes que se siguen ampliamente en varios lenguajes:

- **Formato:** Mantenga la coherencia en el formato de todo el código base. Utilice una herramienta automatizada o un formateador ([negro](#) para

---

Python, [Prettier](#) para JavaScript, etc.) para garantizar un formato consistente en todo el código base.

- **Sangría:** utilice una sangría uniforme (normalmente 2 o 4 espacios) para mejorar la legibilidad.
- **Convenciones de nomenclatura:** Use nombres descriptivos para variables, funciones, clases, etc. Siga una convención de nomenclatura consistente (p. ej., CamelCase, snake\_case) según el lenguaje. Sea cual sea la convención de nomenclatura que elija, cíñase a ella.
- **Comentarios:** Escriba comentarios claros y concisos para secciones de código complejas. Evite comentarios innecesarios que indiquen obviedades. Sea específico y conciso. Escriba lo suficiente para proporcionar el contexto adecuado, pero no exagere. Herramientas como [Mejores comentarios](#) y [Mintlify](#) son excelentes para ayudar a escribir buenos comentarios.
- **Longitud de línea de código:** Limite la longitud de las líneas para mejorar la legibilidad. Muchas guías de estilo recomiendan que las líneas no superen los 80 o 120 caracteres.
- **Longitud de las funciones/métodos:** Mantenga las funciones/métodos breves y centrados en una tarea específica. Las funciones largas deben dividirse en funciones más pequeñas y manejables.
- **Manejo de errores:** Implemente un manejo de errores adecuado y comuníquelos con claridad. El objetivo es identificar y responder adecuadamente a errores y excepciones. Al contar con directrices claras para reportar, registrar y manejar errores, su código se vuelve más confiable y, con el tiempo, más a prueba de errores.
- **Organización de archivos:** Organice el código en secciones o módulos lógicos. Siga una estructura coherente de archivos y directorios.
- **Llaves y paréntesis:** utilice una colocación uniforme de llaves y paréntesis.
- **Espacios en blanco:** Úsalos eficazmente para mejorar la legibilidad del código. Evita espacios en blanco excesivos o innecesarios.

Muchos lenguajes y frameworks tienen sus propias guías de estilo, que a menudo son los estándares preferidos por la comunidad. Algunos ejemplos son:

- **PEP 8:** El estándar de codificación de Python, que limita la longitud de las líneas, especifica la indentación y establece convenciones de nomenclatura.
- **Estándar de codificación de Google:** Una guía detallada con directrices para varios lenguajes, incluyendo Java y Python.
- **WordPress Coding Standards:** Un conjunto de normas de programación para el desarrollo con WordPress, que cubren PHP, HTML, CSS y JavaScript.
- **Rustfmt y Clippy:** Herramientas para el lenguaje Rust que ayudan a aplicar automáticamente los estándares de formato (rustfmt) y a sugerir mejoras (clippy).

Las convenciones de codificación son recomendaciones, mientras que los estándares de codificación son reglas obligatorias y no negociables que el equipo acordó seguir estrictamente.

|            | Estándares de codificación                                            | Convenciones de codificación                                          |
|------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------|
| Definición | Reglas formales y exigibles para la calidad y uniformidad del código. | Prácticas informales y específicas del equipo para lograr coherencia. |
| Formalidad | Alta (estricta y exigible).                                           | Bajo (informal y flexible).                                           |
| Aplicación | Implementado mediante herramientas (linters, pipelines CI).           | Seguido voluntariamente o durante revisiones.                         |
| Objetivo   | Garantizar la calidad, la seguridad y la compatibilidad.              | Promover la coherencia del equipo y las preferencias compartidas.     |

**Bibliografía**